

 MEMOS: A Memory OS for AI System

Zhiyu Li^{1,3}, Shichao Song^{1,8}, Chenyang Xi¹, Hanyu Wang^{1,8}, Chen Tang¹, Simin Niu^{1,8}, Ding Chen¹⁰, Jiawei Yang^{1,8}, Chunyu Li¹, Qingchen Yu^{1,9}, Jihao Zhao^{1,8}, Yezhaohui Wang¹, Peng Liu⁸, Zehao Lin^{1,3}, Pengyuan Wang¹, Jiahao Huo¹, Tianyi Chen^{1,2}, Kai Chen^{1,3}, Kehang Li^{1,2}, Zhen Tao⁸, Junpeng Ren¹, Huayi Lai¹, Hao Wu¹, Bo Tang¹, Zhengren Wang^{7,3}, Zhaoxin Fan⁹, Ningyu Zhang⁵, Linfeng Zhang², Junchi Yan², Mingchuan Yang¹⁰, Tong Xu⁶, Wei Xu⁸, Huajun Chen⁵, Haofeng Wang⁴, Hongkang Yang^{1,3}, Wentao Zhang^{7,†}, Zhi-Qin John Xu^{2,†}, Siheng Chen^{2,†}, Feiyu Xiong^{1,3,†}

¹MemTensor (Shanghai) Technology Co., Ltd., ²Shanghai Jiao Tong University, ³Institute for Advanced Algorithms Research, Shanghai, ⁴Tongji University, ⁵Zhejiang University, ⁶University of Science and Technology of China, ⁷Peking University, ⁸Renmin University of China, ⁹Beihang University, ¹⁰Research Institute of China Telecom

Abstract

Large Language Models (LLMs) have become an essential infrastructure for Artificial General Intelligence (AGI), yet their lack of well-defined memory management systems hinders the development of long-context reasoning, continual personalization, and knowledge consistency. Existing models mainly rely on static parameters and short-lived contextual states, limiting their ability to track user preferences or update knowledge over extended periods. While Retrieval-Augmented Generation (RAG) introduces external knowledge in plain text, it remains a stateless workaround without lifecycle control or integration with persistent representations. Recent work has modeled the training and inference cost of LLMs from a memory hierarchy perspective, showing that introducing an explicit memory layer between parameter memory and external retrieval can substantially reduce these costs by externalizing specific knowledge [1]. Beyond computational efficiency, LLMs face broader challenges arising from how information is distributed over time and context, requiring systems capable of managing heterogeneous knowledge spanning different temporal scales and sources. To address this challenge, we propose MEMOS, a memory operating system that treats memory as a manageable system resource. It unifies the representation, scheduling, and evolution of plaintext, activation-based, and parameter-level memories, enabling cost-efficient storage and retrieval. As the basic unit, a MemCube encapsulates both memory content and metadata such as provenance and versioning. MemCubes can be composed, migrated, and fused over time, enabling flexible transitions between memory types and bridging retrieval with parameter-based learning. MEMOS establishes a memory-centric system framework that brings controllability, plasticity, and evolvability to LLMs, laying the foundation for continual learning and personalized modeling.

Author Legend: †Correspondence

Project Website: <https://memos.openmem.net/>

Code: <https://github.com/MemTensor/MemOS>

1 Introduction

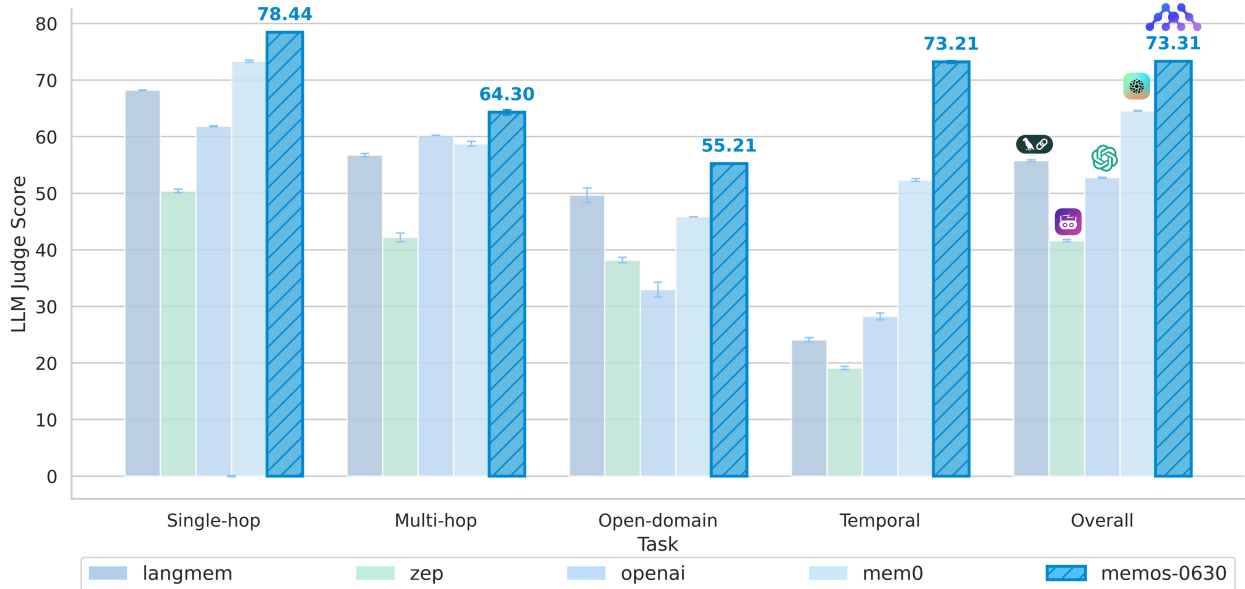


Figure 1 MEMOS achieves state-of-the-art performance across all reasoning tasks. This figure summarizes LLM-Judge scores on the LOCOMO benchmark, covering four task categories (Single-hop, Multi-hop, Open-domain, Temporal Reasoning) and the overall average. MEMOS (MemOS-0630) consistently ranks first in all categories, outperforming strong baselines such as mem0, LangMem, Zep, and OpenAI-Memory, with especially large margins in challenging settings like multi-hop and temporal reasoning. Error bars indicate standard deviation. Full metric breakdown is provided in Table 3.

With the advent of the Transformer architecture and the maturation of self-supervised pretraining, Large Language Models (LLMs) have become the cornerstone of modern NLP. Trained on large-scale corpora, LLMs exhibit near-human performance in open-domain QA, text generation, and summarization tasks [2]. With increasing model size and compute, their capabilities have expanded to structured code generation [3], cross-modal reasoning [4], multi-turn dialogue, and complex planning—positioning LLMs as a leading paradigm toward Artificial General Intelligence (AGI).

Looking ahead, the presence of LLMs, or more generally, AGI systems, will expand vastly in both time and space. Temporally, models will shift from stateless, session-based tools to persistent agents embedded in long-running workflows. Much like humans, they will need to accumulate interaction histories, adapt internal states, and reason over extended contexts. Spatially, LLMs are evolving into foundational intelligence layers across users, platforms, and ecosystems. Whether deployed in cloud services or embedded in enterprise systems, they must support consistency, adaptability, and personalization across users, roles, and tasks. As such omnipresence becomes the norm, a critical challenge emerges: how should knowledge be organized, stored, and retrieved?

With expanding interaction histories, models face a potentially unbounded context space. We anticipate that future LLMs will seek to leverage as much of their accessible temporal and spatial context as possible, to support deeper reasoning, decision-making, and adaptation. No longer reprocessing all past information per inference, they will decide what to retain, compress, discard, or prioritize. In this always-on paradigm, memory becomes a necessity, not an add-on, for maintaining coherent behavior and identity over time. This requires efficient management of large-scale, multi-source information and dynamic scheduling of memory conditioned on context. This motivates a layered memory hierarchy, similar to how OSs manage memory, consisting of working memory, long-term storage, and cold archives, governed by recency, access frequency, and importance. Sharing memory across users and agents requires scoping, permission control, and migratable, reusable representations. These capabilities are vital not only for system efficiency, but for the long-term

sustainability of model-based knowledge evolution.

The management of memory will become model-defined instead of human-defined. Just as deep learning replaced feature engineering, the transition of memory management from hard-coded pipelines (e.g., RAG) to learnable strategies is natural and necessary. Future agents will autonomously decide whether to retrieve memory, summarize interaction into reusable rules, abstract preferences, or transfer knowledge across contexts. In essence, models must take on the responsibility of shaping their own memory architectures and strategies. Yet, existing infrastructures fall short of enabling this shift.

Mainstream LLMs rely on implicit **parameter memory**, encoding knowledge in billions or trillions of model weights. While this approach affords generalization, it suffers from high update cost, poor interpretability, and limited flexibility. Retraining or fine-tuning requires significant computational resources and risks issues such as catastrophic forgetting.

To address this bottleneck, Retrieval-Augmented Generation (RAG) has emerged as a popular augmentation strategy. By incorporating external retrieval modules, RAG allows models to dynamically access fresh information at inference time, enabling augmentation without parameter updates [5–11]. It is now widely deployed in systems such as Copilots [12] and enterprise search [13–16]. Nonetheless, RAG remains fundamentally an “on-the-fly retrieval and transient composition” pipeline, rather than an integrated memory management system. It lacks core memory manageability features such as lifecycle tracking, versioning, and permission-aware scheduling, limiting its ability to support long-term, adaptive knowledge systems. As a result, models continue to exhibit short-memory behavior in multi-turn dialogue, planning, and personalization tasks, struggling to maintain behavioral consistency or long-horizon adaptation.

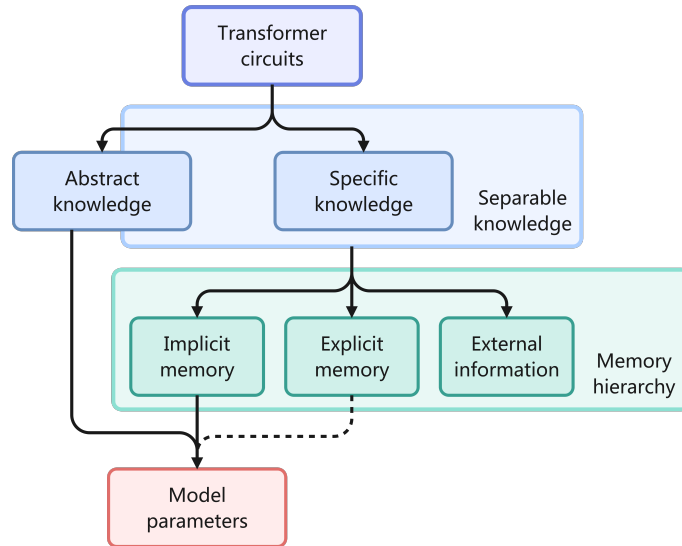


Figure 2 Categorization of LLM knowledge, including the memory hierarchy. The explicit memories, extracted from model activations, lie half-way between raw data and model parameters, so a dotted line is used to indicate that they may or may not be regarded as parameters. Reproduced from [1].

Recent work has shown that the limitations of current memory mechanisms are not incidental, but stem from the architectural absence of explicit and hierarchical memory representations within LLMs. For example, [1] argues that without an intermediate explicit memory layer bridging external retrieval and parametric storage, models become suboptimal in terms of read-write cost, and cannot balance storage cost against retrieval efficiency. This distinction is illustrated in Figure 2, which categorizes knowledge and memory formats and highlights the intermediate role of explicit memory.

From a systems perspective, neither parametric memory nor RAG treats memory as a schedulable and evolvable system resource. This structural gap remains a core bottleneck preventing LLMs from becoming persistent and collaborative intelligent agents. As application scenarios grow more complex, these limitations